

金九求职季之React&工程化面试全知全解

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 年中复盘会（1/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 年中复盘会（2/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 25年下半年面试真题预测及解析](#)
- [📖 阿里/字节面试高频真题解析与9月考点预测](#)
- [📖 阿里&字节9月面试高频真题预测](#)

2. React 进阶用法：HOC & Hooks

2.1 HOC

高阶组件（HOC）是 React 中用于复用组件逻辑的一种高级技巧。HOC 自身不是 React API 的一部分，它是一种基于 React 的组合特性而形成的设计模式。

简单点说，就是组件作为参数，返回值也是组件的函数，它是纯函数，不会修改传入的组件，也不会使用继承来复制其行为。相反，HOC 通过将组件包装在容器组件中来组成新组件。HOC 是纯函数，没有副作用。

2.1.1 使用HOC的原因

1. 抽取重复代码，实现组件复用：相同功能组件复用
2. 条件渲染，控制组件的渲染逻辑（渲染劫持）：权限控制。
3. 捕获/劫持被处理组件的生命周期，常见场景：组件渲染性能追踪、日志打点。

2.1.2 HOC实现方式

2.1.2.1 属性代理

使用组合的方式，将组件包装在容器上，依赖父子组件的生命周期关系来；

1. 返回stateless的函数组件
 2. 返回class组件
- 操作props

Code block

```
1  // 可以通过属性代理，拦截父组件传递过来的props并进行处理。
2
3  // 返回一个无状态的函数组件
4  function HOC(WrappedComponent) {
5      const newProps = { type: 'HOC' };
6      return props => <WrappedComponent {...props} {...newProps}/>;
7  }
8
9  // 返回一个有状态的 class 组件
10 function HOC(WrappedComponent) {
11     return class extends React.Component {
12         render() {
13             const newProps = { type: 'HOC' };
14             return <WrappedComponent {...this.props} {...newProps}/>;
15         }
16     };
17 }
```

- 抽象state

Code block

```
1  // 通过属性代理无法直接操作原组件的state，可以通过props和cb抽象state
2  function HOC(WrappedComponent) {
3      return class extends React.Component {
4          constructor(props) {
5              super(props);
6              this.state = {
7                  name: '',
8              };
9              this.onChange = this.onChange.bind(this);
10         }
11
12         onChange = (event) => {
13             this.setState({
14                 name: event.target.value,
15             })
16         }
17
18         render() {
19             const newProps = {
20                 name: {
21                     value: this.state.name,
22                     onChange: this.onChange,
23                 },
24             };
25             return <WrappedComponent {...this.props} {...newProps} />;
26         }
27     };
28 }
29
30 // 使用
```

```

31  @HOC
32  class Example extends Component {
33    render() {
34      return <input name="name" {...this.props.name} />;
35    }
36  }

```

- 通过props实现条件渲染

Code block

```

1  // 通过props来控制是否渲染及传入数据
2  import * as React from 'react';
3
4  function HOC (WrappedComponent) {
5    return (props) => (
6      <div>
7        {
8          props.isShow ? (
9            <WrappedComponent
10              {...props}
11            />
12          ) : <div>暂无数据</div>
13        }
14      </div>
15    );
16  }
17
18  export default HOC;

```

- 其他元素wrapper传入的组件

Code block

```

1  function withBackgroundColor(WrappedComponent) {
2    return class extends React.Component {
3      render() {
4        return (
5          <div style={{ backgroundColor: '#ccc' }}>
6            <WrappedComponent {...this.props} {...newProps} />
7          </div>
8        );
9      }
10    };
11  }

```

2.1.2.2 反向继承

使用一个函数接受一个组件作为参数传入，并返回一个继承了该传入组件的类组件，且在返回组件的 render() 方法中返回 super.render() 方法

Code block

```

1  const HOC = (WrappedComponent) => {
2    return class extends WrappedComponent {
3      render() {
4        return super.render();
5      }
6    }
7  }

```

1. 允许HOC通过this访问到原组件，可以直接读取和操作原组件的state/ref等；
2. 可以通过super.render()获取传入组件的render，可以有选择的渲染劫持；
3. 劫持原组件生命周期方法

Code block

```
1  function HOC(WrappedComponent){
2    const didMount = WrappedComponent.prototype.componentDidMount;
3
4    // 继承了传入组件
5    return class HOC extends WrappedComponent {
6      async componentDidMount(){
7        // 劫持 WrappedComponent 组件的生命周期
8        if (didMount) {
9          await didMount.apply(this);
10       }
11       ...
12     }
13
14     render(){
15       //使用 super 调用传入组件的 render 方法
16       return super.render();
17     }
18   }
19 }
```

- 读取/操作原组件的state

Code block

```
1  function HOC(WrappedComponent){
2    const didMount = WrappedComponent.prototype.componentDidMount;
3    // 继承了传入组件
4    return class HOC extends WrappedComponent {
5      async componentDidMount(){
6        if (didMount) {
7          await didMount.apply(this);
8        }
9        // 将 state 中的 number 值修改成 2
10       this.setState({ number: 2 });
11     }
12
13     render(){
14       //使用 super 调用传入组件的 render 方法
15       return super.render();
16     }
17   }
18 }
```

- 条件渲染

Code block

```
1  const HOC = (WrappedComponent) =>
2    class extends WrappedComponent {
3      render() {
4        if (this.props.isRender) {
```

```

5         return super.render();
6     } else {
7         return <div>暂无数据</div>;
8     }
9 }
10 }

```

- 修改react树

Code block

```

1  // 修改返回render结果
2  function HigherOrderComponent(WrappedComponent) {
3      return class extends WrappedComponent {
4          render() {
5              const tree = super.render();
6              const newProps = {};
7              if (tree && tree.type === 'input') {
8                  newProps.value = 'something here';
9              }
10             const props = {
11                 ...tree.props,
12                 ...newProps,
13             };
14             const newTree = React.cloneElement(tree, props, tree.props.children);
15             return newTree;
16         }
17     };
18 }

```

2.1.3 属性代理和反向继承对比

1. 属性代理：从“组合”角度出发，有利于从外部操作wrappedComp，可以操作props，或者在wrappedComp 外加一些拦截器（如条件渲染等）；
2. 反向继承：从“继承”角度出发，从内部操作wrappedComp，可以操作组件内部的state，生命周期和render等，功能能加强大；

2.2 Hooks

注意：自定义Hooks本质上还是实现一个函数，关键在于实现逻辑

一般实现效果如：

Code block

```

1  const [ a[, b, c...] ] = useXXX(arg1[, arg2, ...])

```

2.2.1 setTitle hook

Code block

```

1  import { useEffect } from 'react'
2
3  const useTitle = (title) => {
4      useEffect(() => {
5          document.title = title
6      }, [])
7
8      return

```

```

9   }
10
11  export default useTitle
12
13  const App = () => {
14    useTitle('new title')
15    return <div>home</div>
16  }

```

2.2.2 update hook

Code block

```

1  import { useState } from 'react'
2
3  const useUpdate = () => {
4    const [, setFlag] = useState()
5    const update = () => {
6      setFlag(Date.now())
7    }
8    return update
9  }
10
11  export default useUpdate
12
13  // 实际使用
14  const App = (props) => {
15    // ...
16    const update = useUpdate()
17    return <div>
18      {Date.now()}
19      <div><button onClick={update}>update</button></div>
20    </div>
21  }

```

2.2.3 useScroll hooks

Code block

```

1  import { useState, useEffect } from 'react'
2
3  const useScroll = (scrollRef) => {
4    const [pos, setPos] = useState([0,0])
5
6    useEffect(() => {
7      function handleScroll(e){
8        setPos([scrollRef.current.scrollLeft, scrollRef.current.scrollTop])
9      }
10     scrollRef.current.addEventListener('scroll', handleScroll)
11     return () => {
12       scrollRef.current.removeEventListener('scroll', handleScroll)
13     }
14   }, [])
15
16   return pos
17 }
18
19 export default useScroll
20
21 // 用法

```

```
22 import React, { useRef } from 'react'
23 import { useScroll } from 'hooks'
24
25 const Home = (props) => {
26   const scrollRef = useRef(null)
27   const [x, y] = useScroll(scrollRef)
28
29   return <div>
30     <div ref={scrollRef}>
31       <div className="innerBox"></div>
32     </div>
33     <div>{ x }, { y }</div>
34   </div>
35 }
```

2.3 Hooks VS HOC

1. Hook最典型的就是取代掉生命周期中大多数的功能，可以把更相关的逻辑放在一起，而非零散在各个生命周期方法中；
2. 高阶组件可以将外部的属性功能到一个基础 Component 中，更多作为扩展能力的插件（如 react-swipeable-views 中的 autoPlay 高阶组件，通过注入状态化的 props 的方式对组件进行功能扩展，而不是直接将代码写在主库中）；
3. Hook 的写法可以让代码更加紧凑，更适合做 Controller 或者需要内聚的相关逻辑，一般与目标组件内强依赖，HOC 更强调对原先组件能力的扩展；

3. React 源码：Fiber到底是什么

3.1 新老React架构对比

React15:

- Reconciler（协调器）—— 负责找出变化的组件
- Renderer（渲染器）—— 负责将变化的组件渲染到页面上

Reconciler:

<https://zh-hans.reactjs.org/docs/codebase-overview.html#reconcilers>

在React中可以通过this.setState、this.forceUpdate、ReactDOM.render等API触发更新。

每当有更新发生时，Reconciler会做如下工作：

1. 调用函数组件、或class组件的render方法，将返回的JSX转化为虚拟DOM
2. 将虚拟DOM和上次更新时的虚拟DOM对比
3. 通过对比找出本次更新中变化的虚拟DOM
4. 通知Renderer将变化的虚拟DOM渲染到页面上

Renderer:

<https://zh-hans.reactjs.org/docs/codebase-overview.html#renderers>

由于React支持跨平台，所以不同平台有不同的Renderer。我们前端最熟悉的是负责在浏览器环境渲染的Renderer —— ReactDOM。

除此之外，还有：

ReactNative：渲染App原生组件

ReactArt：渲染到Canvas, SVG 或 VML (IE8)

在每次更新发生时，Renderer接到Reconciler通知，将变化的组件渲染在当前宿主环境。

React 15架构的缺点

在Reconciler中，mount的组件会调用mountComponent，update的组件会调用updateComponent。这两个方法会递归更新子组件

递归的缺点：

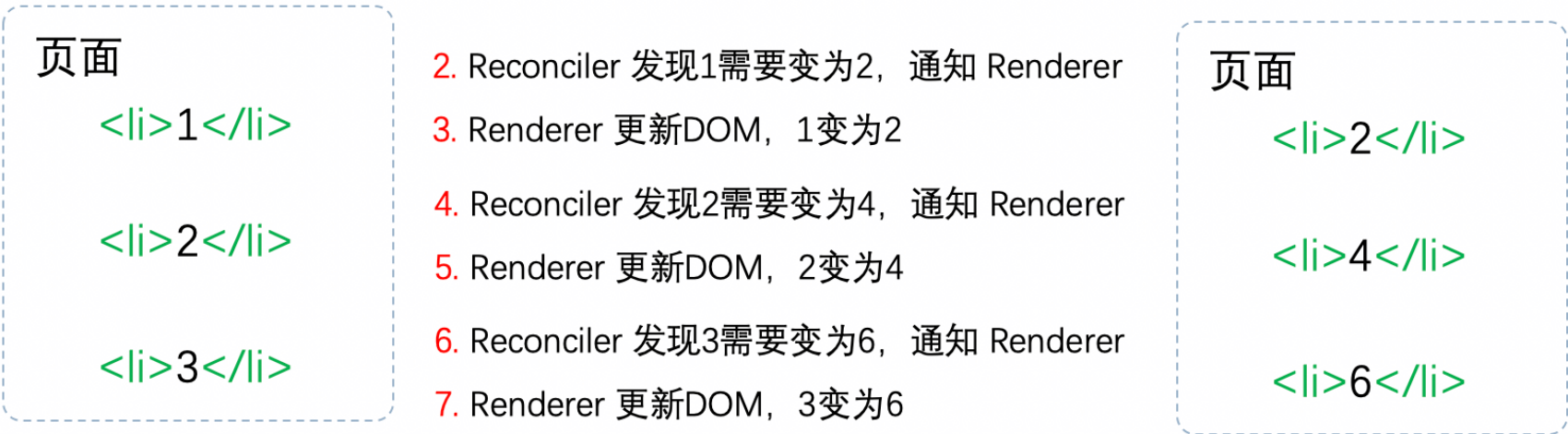
- 1. 当层级很深时，递归更新时间超过了16ms，用户交互就会卡顿
- 2. React 15不支持用可中断的异步更新代替同步的更新

demo：

<https://codesandbox.io/s/fervent-sutherland-pf7sg?file=/src/App.js>

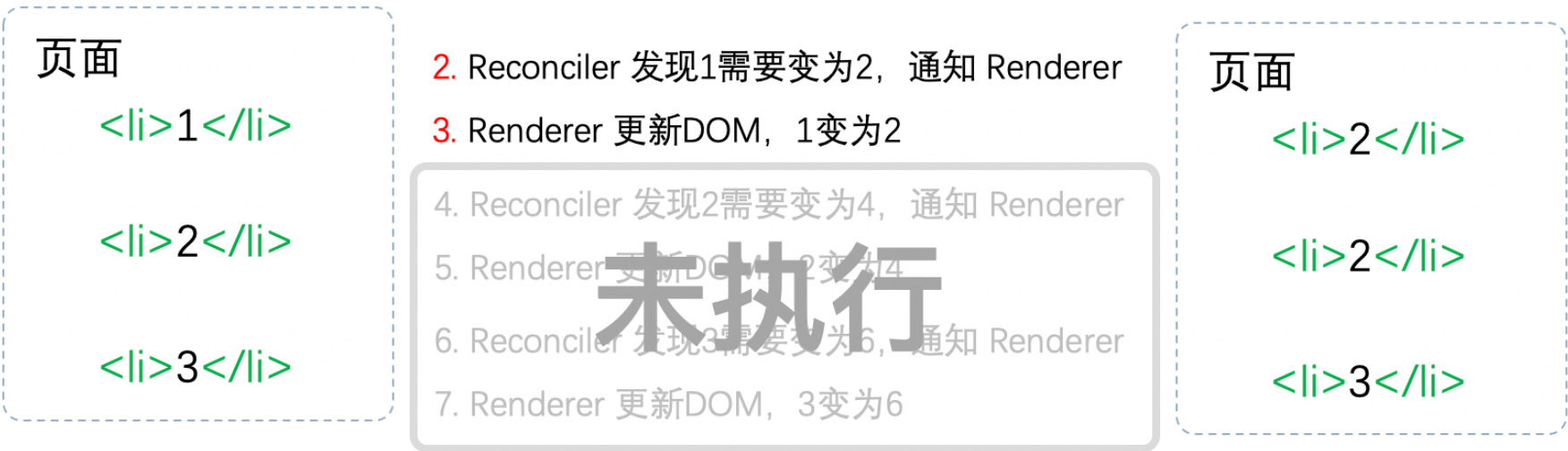
正常：

1. 点击button， state.count 从 1 变为 2



加入同步更新中断

1. 点击button， state.count 从 1 变为 2



React16：

- Scheduler（调度器）—— 调度任务的优先级，高优任务优先进入Reconciler

- Reconciler（协调器）—— 负责找出变化的组件
- Renderer（渲染器）—— 负责将变化的组件渲染到页面上

Scheduler（调度器）：

需要一种机制，当浏览器有剩余时间时通知我们，从而完成任务调度。

部分浏览器已经实现了这个API，这就是[requestIdleCallback](#)。但是由于以下因素，React放弃使用：

- 浏览器兼容性
- 触发频率不稳定，受很多因素影响。比如当我们的浏览器切换tab后，之前tab注册的requestIdleCallback触发的频率会变得很低

基于以上原因，React实现了功能更完备的requestIdleCallbackpolyfill，这就是Scheduler。除了在空闲时触发回调的功能外，Scheduler还提供了多种调度优先级供任务设置。下节课详解

Reconciler（协调器）

我们知道，在React15中Reconciler是递归处理虚拟DOM的。让我们看看[React16的Reconciler](#)

Code block

```
1  // 更新工作从递归变成了可以中断的循环过程。每次循环都会调用shouldYield判断当前是否有剩余时间。
2
3  /** @noinline */
4  function workLoopConcurrent() {
5    // Perform work until Scheduler asks us to yield
6    while (workInProgress !== null && !shouldYield()) {
7      workInProgress = performUnitOfWork(workInProgress);
8    }
9  }
```

Q：如何处理中断更新时DOM渲染不完全？

在React16中，Reconciler与Renderer不再是交替工作。当Scheduler将任务交给Reconciler后，Reconciler会为变化的虚拟DOM打上代表增/删/更新的标记

标记类型详情：<https://github.com/facebook/react/blob/1fb18e22ae66fdb1dc127347e169e73948778e5a/packages/react-reconciler/src/ReactSideEffectTags.js>

Code block

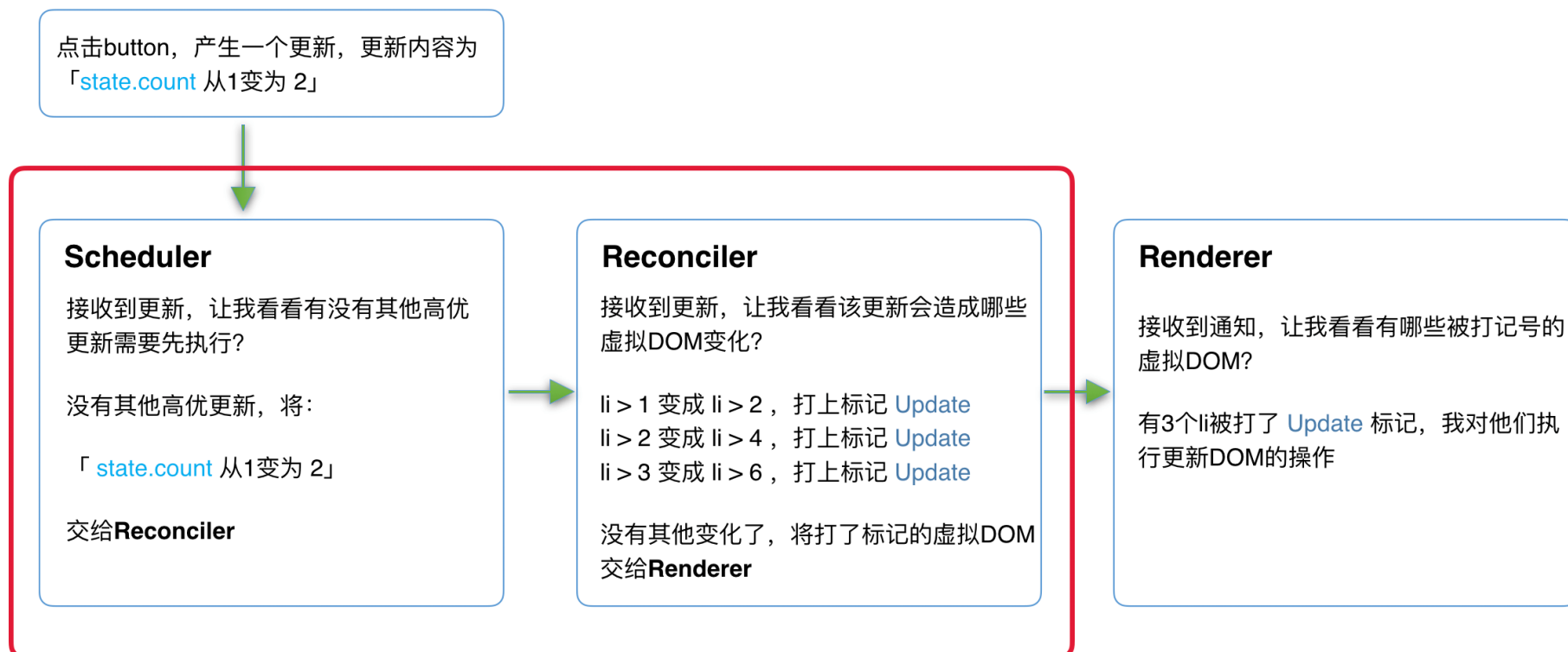
```
1  export const Placement = /*           / 0b00000000000010;
2  export const Update = /               / 0b00000000000100;
3  export const PlacementAndUpdate = /   / 0b00000000000110;
4  export const Deletion = /             */ 0b0000000001000;
```

整个Scheduler与Reconciler的工作都在内存中进行。只有当所有组件都完成Reconciler的工作，才会统一交给Renderer。

详细官方介绍：<https://github.com/facebook/react/blob/1fb18e22ae66fdb1dc127347e169e73948778e5a/packages/react-reconciler/src/ReactSideEffectTags.js>

Renderer（渲染器）

Renderer根据Reconciler为虚拟DOM打的标记，同步执行对应的DOM操作。



其中红框中的步骤随时可能由于以下原因被中断:

- 有其他更高优任务需要先更新
- 当前帧没有剩余时间

由于红框中的工作都在内存中进行, 不会更新页面上的DOM, 所以即使反复中断, 用户也不会看见更新不完全的DOM

同时, 由于Scheduler和Reconciler都是平台无关的, 所以React为他们分别单独发布了一个包

3.2 Fiber架构详解

fiber的核心思路: 在react中遵循代数效应 (algebraic effects)

代数效应是函数式编程中的一个概念, 用于将副作用从函数调用中分离。

Code block

```
1 function getTotalPicNum(user1, user2) {
2   const picNum1 = getPicNum(user1);
3   const picNum2 = getPicNum(user2);
4
5   return picNum1 + picNum2;
6 }
```

假设getPickNum需要异步请求: async await?

- 破坏了上下文的一致性, 需要调用它的函数也时async

假如有一个类似的try...catch语法 -----try...handle、perform、resume

Code block

```
1 function getPicNum(name) {
2   const picNum = perform name;
3   return picNum;
4 }
5
6 try {
```

```

7   getTotalPicNum('xianzao', 'houwan');
8 } handle (who) {
9   switch (who) {
10    case 'xianzao':
11      resume with 230;
12    case 'houwan':
13      resume with 122;
14    default:
15      resume with 0;
16  }
17 }

```

代数效应：将副作用（例子中为请求图片数量）从函数逻辑中分离，使函数关注点保持纯粹，也就是不用关心是同步还是异步

Example：Hooks，不用关心useState中state是如何保存变化的，我们只需要使用即可

Q：为什么不使用generator？

- 类似async，会影响上下文；
- generator的执行状态时上下文关联的

Code block

```

1  function* doWork(A, B, C) {
2    var x = doExpensiveWorkA(A);
3    yield;
4    var y = x + doExpensiveWorkB(B);
5    yield;
6    var z = y + doExpensiveWorkC(C);
7    return z;
8  }

```

- 单一任务中断执行：ok；
- 中间有高优先级的任务：但是当我们考虑“高优先级任务插队”的情况，如果此时已经完成doExpensiveWorkA与doExpensiveWorkB计算出x与y。此时B组件接收到一个高优更新，由于Generator执行的中间状态是上下文关联的，所以计算y时无法复用之前已经计算出的x，需要重新计算；
- 使用全局变量保存之前的中间状态，引入新的复杂度；

React Fiber

1. 定义：React内部实现的一套状态更新机制。支持任务不同优先级，可中断与恢复，并且恢复后可以复用之前的中间状态。
2. 功能：
 - a. 作为架构来说，之前React15的Reconciler采用递归的方式执行，数据保存在递归调用栈中，所以被称为stack Reconciler。React16的Reconciler基于Fiber节点实现，被称为Fiber Reconciler；
 - b. 作为静态的数据结构来说，每个Fiber节点对应一个React element，保存了该组件的类型（函数组件/类组件/原生组件...）、对应的DOM节点等信息；
 - c. 作为动态的工作单元来说，每个Fiber节点保存了本次更新中该组件改变的状态、要执行的工作（需要被删除/被插入页面中/被更新...）；

react fiber node定义

地址：<https://github.com/facebook/react/blob/1fb18e22ae66fdb1dc127347e169e73948778e5a/packages/react-reconciler/src/ReactFiber.new.js#L117>

```
1 function FiberNode(  
Code block  
2   tag: WorkTag,  
3   pendingProps: mixed,  
4   key: null | string,  
5   mode: TypeOfMode,  
6 ) {  
7   // Instance, 静态节点的数据结构属性  
8   this.tag = tag;  
9   this.key = key;  
10  this.elementType = null;  
11  this.type = null;  
12  this.stateNode = null;  
13  
14  // Fiber, 用来链接其他fiber节点形成的fiber树  
15  this.return = null;  
16  this.child = null;  
17  this.sibling = null;  
18  this.index = 0;  
19  
20  this.ref = null;  
21  
22  // 作为动态的工作单元的属性  
23  this.pendingProps = pendingProps;  
24  this.memoizedProps = null;  
25  this.updateQueue = null;  
26  this.memoizedState = null;  
27  this.dependencies = null;  
28  
29  this.mode = mode;  
30  
31  this.effectTag = NoEffect;  
32  this.subtreeTag = NoSubtreeEffect;  
33  this.deletions = null;  
34  this.nextEffect = null;  
35  
36  this.firstEffect = null;  
37  this.lastEffect = null;  
38  
39  // 作为调度优先级的属性  
40  this.lanes = NoLanes;  
41  this.childLanes = NoLanes;  
42  
43  // 指向该fiber在另一次更新时对应的fiber  
44  this.alternate = null;  
45  
46  if (enableProfilerTimer) {  
47    // Note: The following is done to avoid a v8 performance cliff.  
48    //  
49    // Initializing the fields below to smis and later updating them with  
50    // double values will cause Fibers to end up having separate shapes.  
51    // This behavior/bug has something to do with Object.preventExtension().  
52    // Fortunately this only impacts DEV builds.  
53    // Unfortunately it makes React unusably slow for some applications.  
54    // To work around this, initialize the fields below with doubles.  
55    //  
56    // Learn more about this here:  
57    // https://github.com/facebook/react/issues/14365  
58    // https://bugs.chromium.org/p/v8/issues/detail?id=8538  
59    this.actualDuration = Number.NaN;  
60    this.actualStartTime = Number.NaN;  
61    this.selfBaseDuration = Number.NaN;  
62    this.treeBaseDuration = Number.NaN;
```

```

63
64     // It's okay to replace the initial doubles with smis after initialization.
65     // This won't trigger the performance cliff mentioned above,
66     // and it simplifies other profiler code (including DevTools).
67     this.actualDuration = 0;
68     this.actualStartTime = -1;
69     this.selfBaseDuration = 0;
70     this.treeBaseDuration = 0;
71   }
72
73   if (__DEV__) {
74     // This isn't directly used but is handy for debugging internals:
75     this._debugID = debugCounter++;
76     this._debugSource = null;
77     this._debugOwner = null;
78     this._debugNeedsRemount = false;
79     this._debugHookTypes = null;
80     if (!hasBadMapPolyfill && typeof Object.preventExtensions === 'function') {
81       Object.preventExtensions(this);
82     }
83   }
84 }

```

- 架构层面

Code block

```

1  // 指向父级Fiber节点
2  this.return = null;
3  // 指向子Fiber节点
4  this.child = null;
5  // 指向右边第一个兄弟Fiber节点
6  this.sibling = null;
7
8  表示的组件结构
9  function App() {
10   return (
11     <div>
12       i am
13       <span>xianzao</span>
14     </div>
15   )
16 }

```

Q：为什么指向的父节点是return而不是parent？

因为作为一个工作单元，return指节点执行完completeWork后会返回的下一个节点。子Fiber节点及其兄弟节点完成工作后会返回其父级节点，所以用return指代父级节点

- 作为静态数据结构

Code block

```

1  // Fiber对应组件的类型 Function/Class...
2  this.tag = tag;
3  // key属性
4  this.key = key;
5  // 大部分情况同type，某些情况不同，比如FunctionComponent使用React.memo包裹
6  this.elementType = null;
7  // 对于 FunctionComponent，指函数本身，对于ClassComponent，指class
8  this.type = null;

```

```
9  // Fiber对应的真实DOM节点
10 this.stateNode = null;
```

- 作为动态工作单元

记录更新相关的信息，主要是updateQueue

Q：React Fiber 如何更新DOM？

使用“双缓存”

在内存中绘制当前的fiber dom，绘制完毕后直接替换上一帧的fiber dom，由于省去了两帧替换间的计算时间，不会出现从白屏到出现画面的闪烁情况

在React中最多会同时存在两棵Fiber树。当前屏幕上显示内容对应的Fiber树称为current Fiber，正在内存中构建的Fiber树称为workInProgress Fiber，两者通过alternate连接

Code block

```
1  currentFiber.alternate === workInProgressFiber;
2  workInProgressFiber.alternate === currentFiber;
```

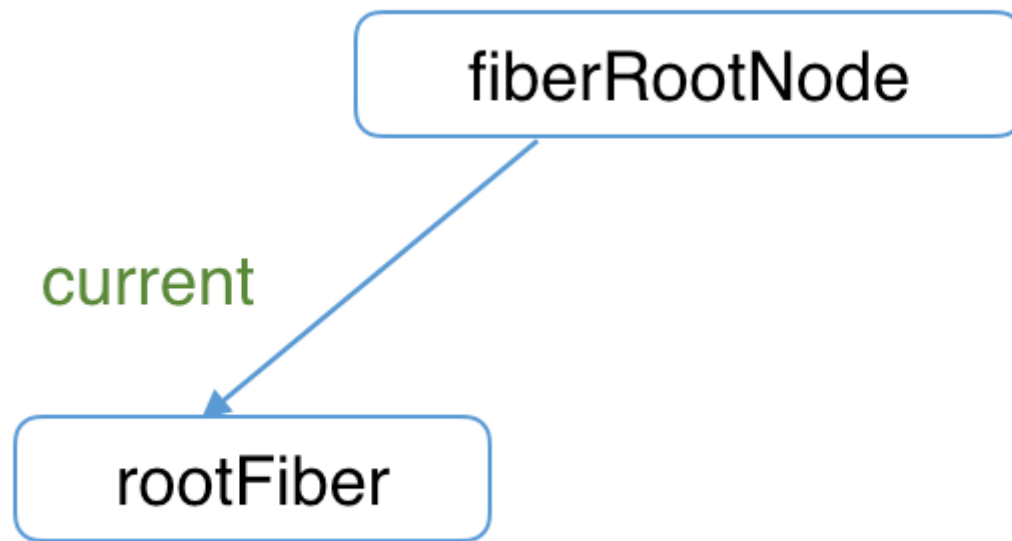
React应用的根节点通过current指针指向不同的fiber dom切换，当update时，workInProgressFiber render完成后会跟currentFiber 替换，下一次更新会将当前currentFiber（上一次的workInProgressFiber）替换

Code block

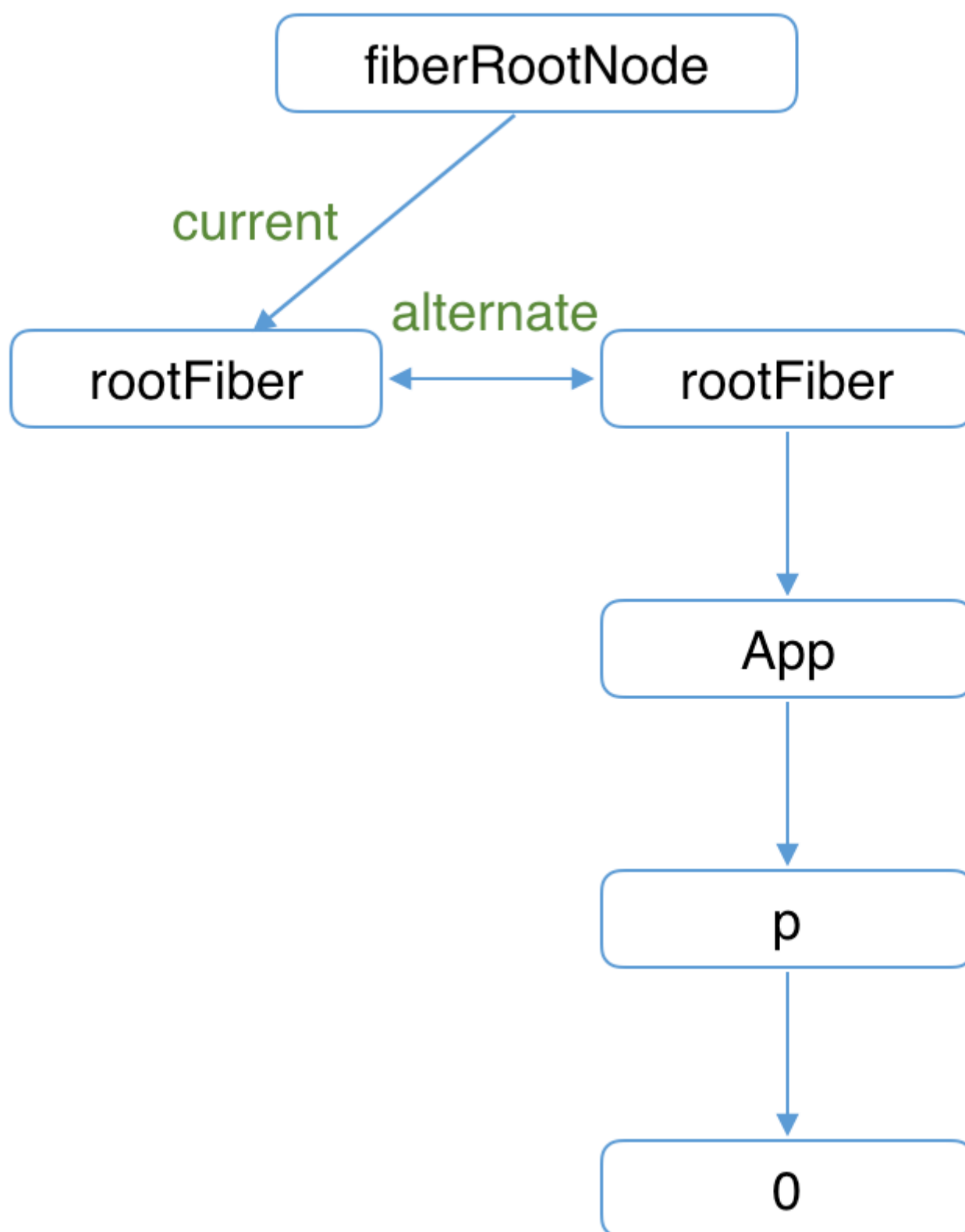
```
1  function App() {
2    const [num, add] = useState(0);
3    return (
4      <p onClick={() => add(num + 1)}>{num}</p>
5    )
6  }
7
8  ReactDOM.render(<App/>, document.getElementById('root'));
```

- mount

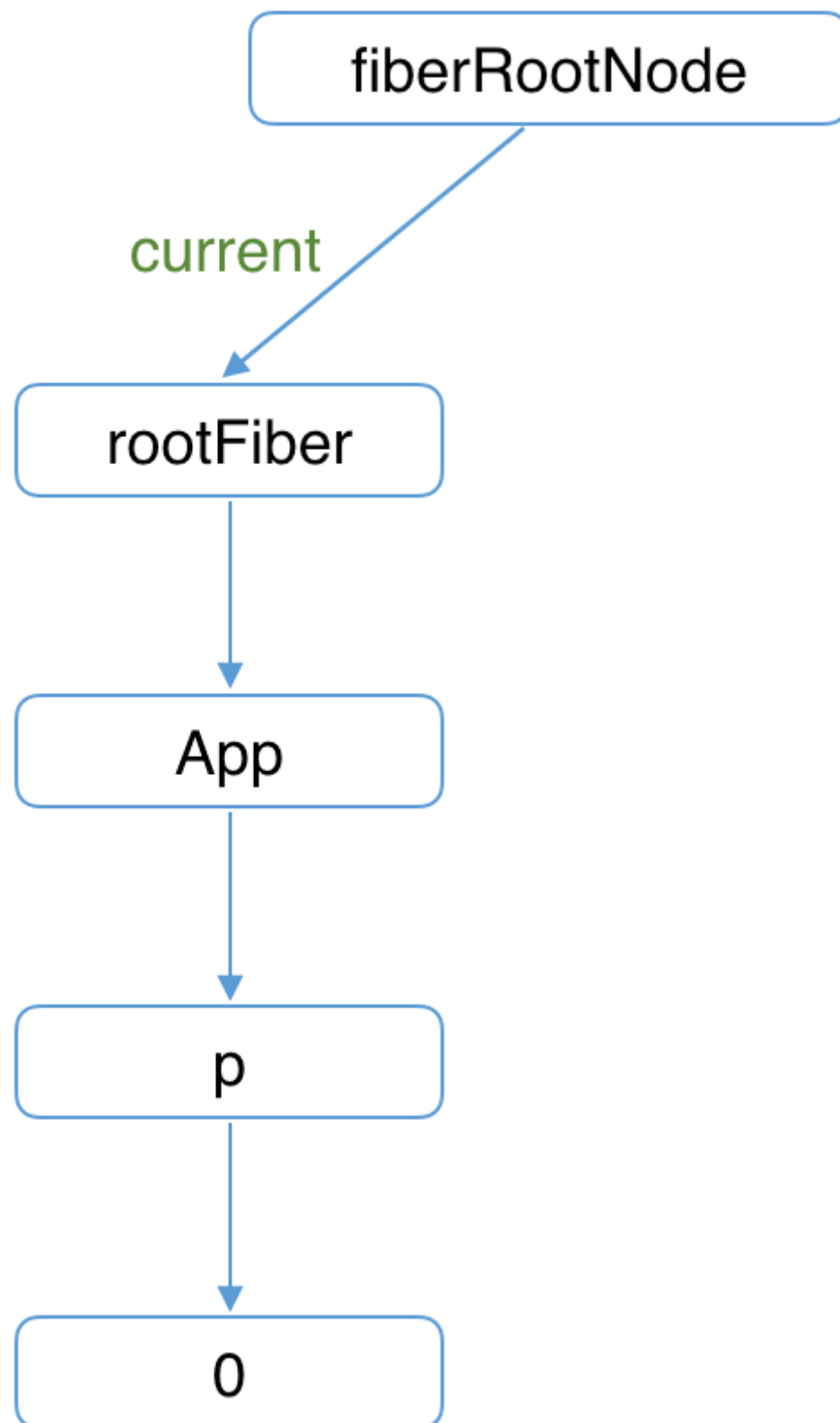
1. 首次执行ReactDOM.render会创建fiberRootNode（源码中叫fiberRoot）和rootFiber。其中fiberRootNode是整个应用的根节点，rootFiber是<App/>所在组件树的根节点；
 - a. 区分fiberRootNode与rootFiber：因为在应用中我们可以多次调用ReactDOM.render渲染不同的组件树，他们会拥有不同的rootFiber。但是整个应用的根节点只有一个，那就是fiberRootNode；
 - b. fiberRootNode的current会指向当前页面上已渲染内容对应Fiber树，即current Fiber；
 - c. 因为是首次渲染，此时页面还没有挂在所有的DOM，所以rootFiber还没有子fiber dom



2. render阶段，根据组件返回的JSX在内存中依次创建Fiber节点并连接在一起构建Fiber树，被称为workInProgress Fiber；
 - a. 在构建workInProgress Fiber树时会尝试复用current Fiber树中已有的Fiber节点内的属性（后续的diff），在首屏渲染时只有rootFiber存在对应的current fiber（即rootFiber.alternate）

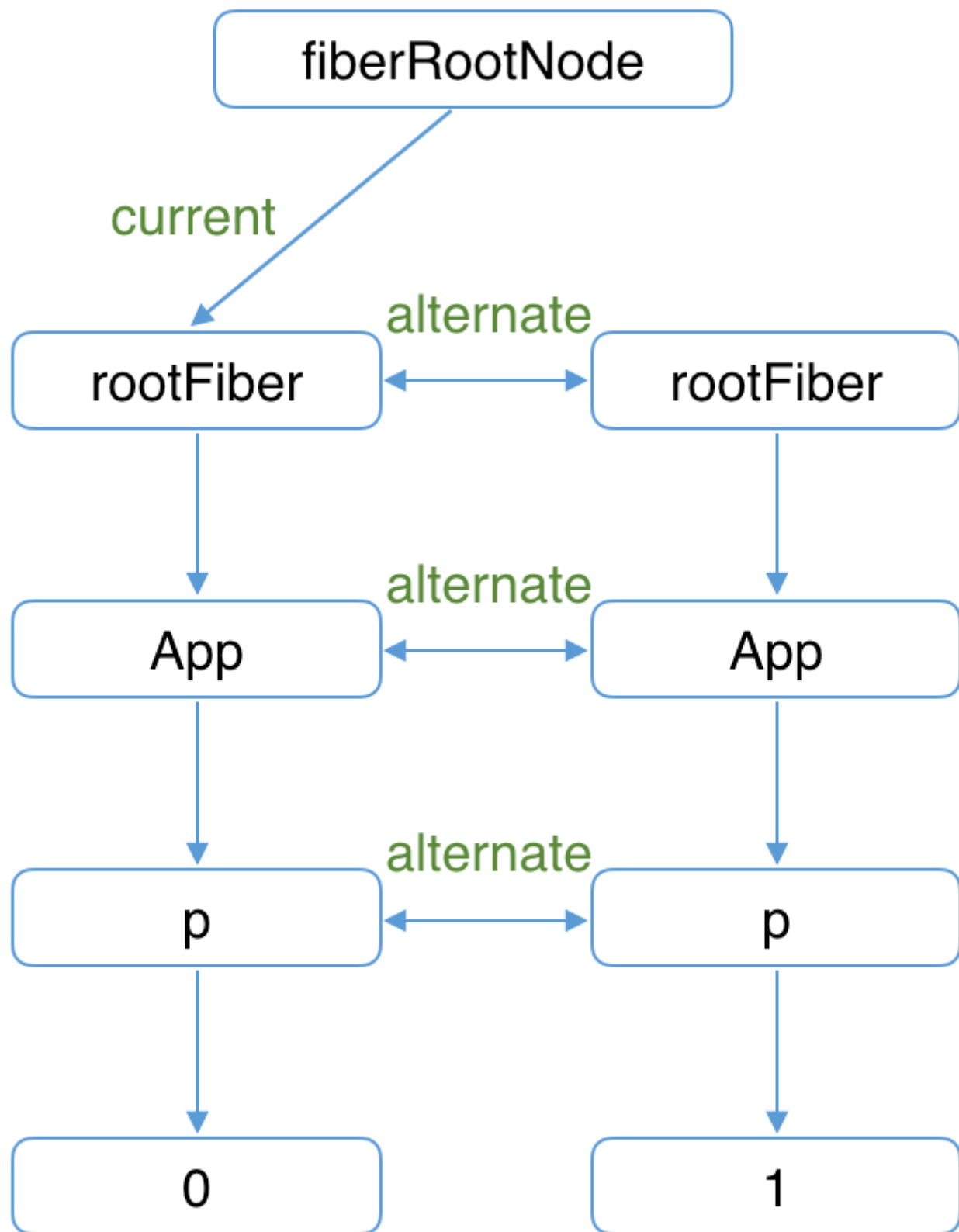


3. alternate阶段：此时workInProgress fiber已经构建完成，fiberRootNode的current指向了workInProgress fiber



- update

1. 假设p元素更新，这会开启一次新的render阶段并构建一棵新的workInProgress Fiber 树，且会尽可能服用显有的current Fiber



4. 9~10月如何平稳收获offer?



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

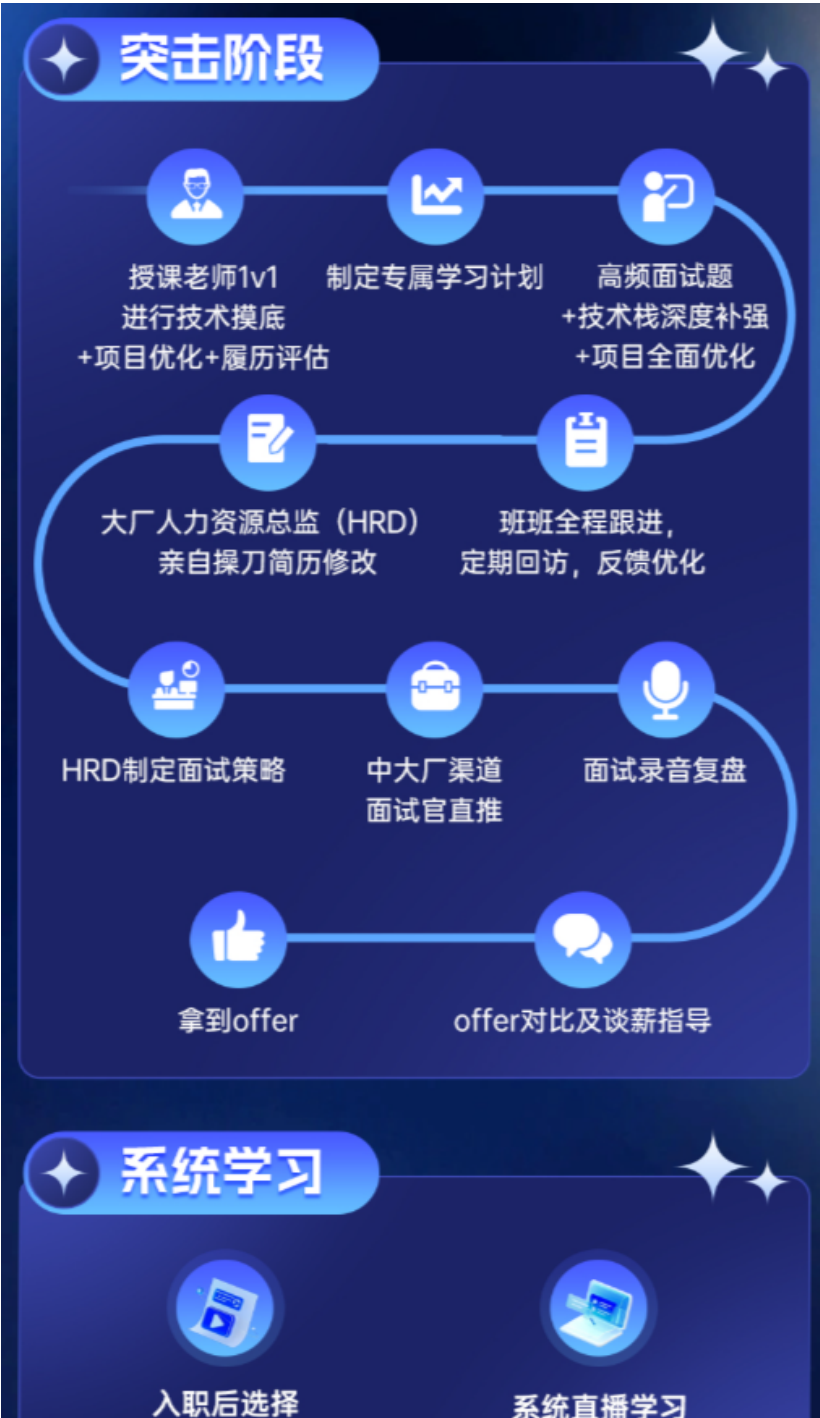
WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7





4.1 专业一对一

近期一对一

- 📅 (2025-05-06) xxx-2.5年-React-24K
- 📅 (2025-05-07) xxx-1年-React-15k
- 📅 (2025-05-25) xxx-7年-Vue-20k

报名学员专享

📖 13. 如何写出高质量的简历（For 付费学员）

4.2 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

5. React 工程化：React Compiler & 性能优化

5.1 React Compiler

🔗 <https://react.dev/learn/react-compiler>

React Compiler的原理是通过静态分析代码，并在编译时将性能优化操作（如 `useMemo` 和 `useCallback` 的自动应用）转移到运行时之前，从而实现更精细粒度的组件更新优化。它使用纯JavaScript，并理解React规则，能够在不重写现有代码的情况下，自动识别并缓存依赖项，精确地最小化更新，从而减少不必要的渲染。

工作原理详解

1. 静态分析：

- React Compiler在编译阶段对React组件进行静态分析，理解代码结构和数据依赖关系。
- 它分析组件内部变量、属性和函数的变化，并确定哪些部分在数据改变时需要重新渲染。

2. 编译时优化：

- 与手动添加 `useMemo` 和 `useCallback` 不同，编译器会根据静态分析的结果，自动在编译时生成相应的优化代码。
- 例如，当识别到某个函数在组件内是匿名的并且其依赖项不变时，编译器会将其包裹在 `useCallback` 中，防止其在每次渲染时都重新创建。

3. 最小化更新：

- 编译后的组件能够实现更细粒度的更新，只渲染那些实际依赖于变化数据的部分。
- 这意味着，即使父组件发生渲染，如果子组件的依赖项没有变化，那么该子组件就不会被重新渲染，从而显著提升性能。

4. 纯JavaScript和无代码修改：

- React Compiler使用纯JavaScript语言开发，并能够理解React的规则，使得开发者无需重写现有的代码即可应用这些优化。

5. 集成到开发流程：

- 它提供了一个ESLint插件，该插件可以在编辑器中直接显示编译器的分析结果，帮助开发者在开发过程中提高代码质量和性能意识。

6. 性能优化

定量指标：

- Largest Contentful Paint : 最大内容绘制，测量加载性能。理想值 < 2.5 秒。
- First Input Delay : 首次输入延迟，测量交互性。理想值 < 100 毫秒。
- Cumulative Layout Shift : 累积布局偏移，测量视觉稳定性。理想值 < 0.1。

6.1 编译构建时优化

1. 体积优化：terser 压缩、混淆

2. 资源优化

- 图片：webp等、srcset 基于dpi尺寸
- 字体：预加载关键字体: `<link rel="preload" as="font">`。
- 懒加载：script defer。
- CDN

3. 减少 HTTP 请求

- 文件合并:** 在 HTTP/1.1 时代常用，但在 HTTP/2+ 下需权衡（多路复用减少了合并的必要性，合并可能影响缓存粒度）。小图标合并成雪碧图仍有价值。
- 内联关键 CSS/JS:** 首屏内容 & 骨架屏 所必需的小段 CSS/JS 直接内联在 HTML 中，避免阻塞渲染的请求。

4. 利用浏览器缓存

6.2 运行时优化

1. 优化 CSS

- 避免深层嵌套选择器: 降低匹配复杂度。
- 减少过度使用昂贵的属性: 如 `box-shadow` , `border-radius` , `filter`

2. 优化 JavaScript 执行

- 避免长任务: 将耗时 JS 任务分解成小块 (使用 `setTimeout` , `requestAnimationFrame` , `requestIdleCallback` , Web Workers)。
- 防抖和节流: 控制高频事件 (resize, scroll, input) 的处理频率。
- Web Workers: 将复杂的计算任务移出主线程，避免阻塞 UI 渲染和交互响应。
- 优化事件处理: 使用事件委托，及时移除不再需要的事件监听器。

- **强缓存 (Cache-Control, Expires):** 为静态资源设置较长的过期时间 (e.g., `max-age=31536000`, `immutable`)。
- **协商缓存 (ETag, Last-Modified):** 确保用户代理能有效验证资源是否过期。
- **Service Worker:** 实现更精细的缓存策略（网络优先、缓存优先等），支持离线访问。PWA 的核心。

5. 减少资源体积

- Code Splitting
- Tree Shaking
- Webpack Bundle Analyzer。

6. 预加载、预连接、预获取

- `<link rel="preload">` : ** 强制浏览器提前加载关键资源 (字体、首屏图片、关键 JS/CSS)。
- `<link rel="preconnect">` : ** 提前建立与关键第三方源 (CDN, API 域名, 分析服务) 的连接 (DNS 查找、TCP 握手、TLS 协商)。
- `<link rel="prefetch">` : ** 在浏览器空闲时预取用户下一步可能需要的资源 (如下一页的内容、非首屏图片)。
- `<link rel="dns-prefetch">` : ** 提前解析域名 DNS (比 `preconnect` 更轻量)。

7. HTTP/2 或 HTTP/3

- 确保服务器和 CDN 支持 HTTP/2 (多路复用、头部压缩、服务器推送 - 需谨慎使用) 或更先进的 HTTP/3 (基于 QUIC, 改进弱网性能)。

- e. 虚拟列表: 对于超长列表, 仅渲染可视区域内的项 (`react-window`, `vue-virtual-scroller`)。

3. 优化渲染流程:

- a. 减少重排和重绘: 集中修改 DOM 样式 (使用 `class` 切换), 避免逐个修改。离线操作 DOM (使用 `DocumentFragment`, `cloneNode`, 隐藏元素修改完再显示)。
- b. GPU 加速: 对动画元素使用 `transform` 和 `opacity` 属性, 它们通常在合成层处理, 避免触发重排重绘。

4. 框架优化: 避免不必要的渲染

- a. React: 使用 `React.memo`, `PureComponent`, `useMemo`, `useCallback`。优化 Context 的使用。
- b. Vue: 利用 `v-once`, `v-memo` (Vue 3), 计算属性缓存, 合理设计组件。

6.3 用户体验

1. 骨架屏
2. 占位符
3. 进度条
4. 交互优化 Hover click etc
5. 资源缓存 LRU